Small Vertex Cut in the PRAM Model

Vasileios-Christos Xanthopoulos*

Abstract

The vertex cut problem asks for a minimum set of vertices whose removal disconnects a given graph. It has applications in fault-tolerant network design, parallel computation, and distributed systems. We study the problem in the PRAM model. Building on the sequential algorithm of Nanongkai et al. for small vertex connectivity and the parallel reachability algorithm of Sidford et al., we present a parallel randomized algorithm for finding a separator of size less than k. Our algorithm gives a work–depth tradeoff controlled by a parameter α. For $k = O(\sqrt{n})$, it achieves $\widetilde{O}(mk^2 + m^2/\alpha)$ work and $\max\{n^{1/2+o(1)}, k\alpha\}$ depth with high probability. For constant k, we also give a vertex-sampling variant that improves the work in dense regimes. We conclude with open problems.

1 Introduction

The vertex cut problem is a classical problem in graph theory and network reliability. Given a graph $G = (V, E)$, a *vertex cut* or *separator* is a set $S \subseteq V$ whose removal disconnects the graph into multiple connected components. The problem is fundamental in fault-tolerant network design, parallel computation, and distributed systems.

In this paper, we study the small vertex cut problem in the PRAM model. Prior parallel algorithms for vertex connectivity, such as the classical results by Cheriyan and Thurimella [1], achieve polynomial work but suffer from $\tilde{\Omega}(n)$ depth or worse in the PRAM model. We bridge this gap by building on two recent lines of work: the sequential small vertex connectivity algorithm of Nanongkai et al. [3], which broke the quadratic barrier for small k, and the parallel reachability algorithm of Sidford et al. [4], which uses hopsets to achieve almost linear work and $n^{1/2+o(1)}$ depth. Our main contribution is a parallel algorithm that combines the two techniques and yields a rigorous work–depth tradeoff.

The algorithm follows the high-level structure of the sequential algorithm: it distinguishes between balanced and unbalanced separations. In the balanced case, it uses a parallel version of Ford–Fulkerson; in the unbalanced case, it uses a local vertex cut procedure, replacing depth-heavy DFS traversals by parallel reachability when the relevant side of the cut is large.

Main results. Our first main result is the following.

Theorem 1.1 (Parallel Vertex Cut)**.** *There exists a parallel randomized (Monte Carlo) algorithm that, given a directed graph $G = (V, E)$, $k = O(\sqrt{n})$, and $\alpha \in [\sqrt{n}, n]$, if G is not k-connected then* w.h.p. *finds a separator S of size less than k. If G is k-connected then the algorithm always returns $\perp$. The algorithm takes $\widetilde{O}(mk^2 + \frac{m^2}{\alpha})$ work and $\max\{n^{1/2+o(1)}, k\alpha\}$ depth.*

*National Technical University of Athens. Email. This work was done in collaboration with Yonngang Jiang (MPI) and supervised by Danupon Nanongkai (MPI).

For constant k, we also obtain a vertex-sampling variant.

Theorem 1.2 (Parallel Vertex Cut with Constant k)**.** *There exists a parallel randomized (Monte Carlo) algorithm that, given a directed graph $G = (V, E)$, $k = O(1)$, and $\alpha \in [\sqrt{n}, n]$, if G is not k-connected then* w.h.p. *finds a separator S of size less than k. If G is k-connected then the algorithm always returns $\perp$. The algorithm takes $\widetilde{O}(\frac{mn}{\alpha})$ work and $O(\alpha)$ depth.*

2 Preliminaries

2.1 PRAM and work–depth

We work in the CREW PRAM model. A parallel algorithm is analyzed by its *work* W, the total number of elementary operations, and its *depth* D, the length of the longest dependency chain. Work corresponds to sequential time T_1, while depth corresponds to parallel time with an unbounded number of processors, T_∞. An event occurs *with high probability* (w.h.p.) if it holds with probability at least $1 - n^{-c}$ for some constant $c > 0$.

2.2 Vertex cuts

Let $G = (V, E)$ be a directed graph. For a set $X \subseteq V$, let $N_G^{\text{out}}(X)$ denote its out-neighborhood.

Definition 2.1 (Separation triplet)**.** *A partition (L, S, R) of V is a* separation triplet *if $L, R \neq \emptyset$ and there is no edge from L to R. Equivalently, $N_G^{\text{out}}(L) = S$. We call S a* separator*.*

The graph is *k-connected* if it has no separation triplet with $|S| < k$. For a separation triplet (L, S, R), define

$$E^*(L, S) = E(L, L) \cup E(L, S) \cup E(S, L), \qquad \text{vol}_G^*(L) = |E^*(L, S)|.$$

The quantities $\text{vol}_G^*(R)$ and $E^*(R, S)$ are defined analogously, and $m = \text{vol}_G^*(L) + |E(S, S)| + \text{vol}_G^*(R)$.

2.3 Reduction to directed edge connectivity

We reduce vertex cut to directed edge cut using the standard split-graph transformation. For each $v \in V$, create vertices v_{in} and v_{out} with a unit-capacity edge $(v_{\text{in}}, v_{\text{out}})$. For each edge $(u, v) \in E$, add the edge $(u_{\text{out}}, v_{\text{in}})$. The source becomes s_{in} and the sink t_{out}. The transformation preserves vertex cuts bijectively with edge cuts of equal size, so we may work directly with directed edge cuts throughout.

2.4 Hopsets and shortcuts

A *shortcut* is an edge (u, v) added to G such that u already reaches v in G. Adding shortcuts preserves reachability but reduces distances. A *hopset* is a collection of shortcuts. The parallel reachability algorithm of Sidford et al. [4] constructs a hopset of size $\widetilde{O}(n)$ that reduces the diameter of any n-node digraph to $n^{1/2+o(1)}$ w.h.p., while running in $\widetilde{O}(m)$ work and $n^{1/2+o(1)}$ depth.

3 Building blocks

3.1 Parallel reachability

The algorithm of Sidford et al. [4] follows Fineman's shortcutting framework [2]. At each level of recursion, it selects a set S of random *shortcutters*, where $|S|$ grows by a factor of k per level. For each $v \in S$, it adds shortcut edges from v to all its descendants and from all its ancestors to v. Each remaining vertex receives a label recording its relationship to each shortcutter; vertices sharing the same label set form a subproblem, and the algorithm recurses. To parallelize, full BFS traversals are replaced by depth-restricted searches of radius κD, where D is the diameter bound.

Theorem 3.1 (Parallel reachability [4])**.** *There exists a parallel algorithm that, given an n-node m-edge digraph and a source vertex s, computes the set of vertices reachable from s and a set of paths to them in $\widetilde{O}(m)$ work and $n^{1/2+o(1)}$ depth, with high probability in n.*

3.2 Sequential local vertex cut

The sequential local vertex cut algorithm of Nanongkai et al. [3] takes a seed vertex x, a volume threshold ν, and a cut size k. It performs a DFS from x exploring up to $k\nu$ edges. If the DFS terminates early, the explored set has out-neighborhood of size $< k$. Otherwise, it samples a random edge from the DFS tree, reverses the path from x to that edge, and repeats, reducing the out-degree of the current set by one each time. After k iterations, the algorithm returns $\perp$, and an outer loop boosts the success probability.

Theorem 3.2 (Local vertex cut)**.** *There exists a randomized algorithm* LocalVC(G, x, ν, k) *that, given a directed graph G, a seed vertex x, and integers ν, k, returns a set L' containing x with $|N_G^{\text{out}}(L')| < k$ and $\text{vol}(L') < \nu$ if such a set exists, and returns $\perp$ otherwise. Its sequential running time is $\widetilde{O}(\nu k^2)$, and its parallel depth if implemented by DFS is $O(\nu k)$.*

3.3 Sequential small vertex connectivity

The sequential algorithm of [3] proceeds in two phases. In the balanced phase, it samples pairs of edges and tests the sampled endpoints for a small cut using Ford–Fulkerson. In the unbalanced phase, it samples edges to hit the small side of the cut and applies LocalVC around the sampled vertex. The overall running time is $\widetilde{O}(mk^2)$.

4 The parallel algorithm

4.1 Overview

Suppose G is not k-connected, so there is a separation triplet (L, S, R) with $|S| < k$. We assume without loss of generality that $\text{vol}^*_G(L) \le \text{vol}^*_G(R)$ and distinguish two cases. When $\text{vol}^*_G(L) \ge m/k$ (balanced), we sample random edge pairs and test for a small cut using a parallel Ford–Fulkerson subroutine. When $\text{vol}^*_G(L) < m/k$ (unbalanced), we sample edges to hit the small side and run a local vertex cut procedure around the sampled vertex.

4.2 Modified Ford–Fulkerson

The original Ford-Fulkerson algorithm uses BFS to find augmenting paths, which has depth proportional to the graph diameter (up to $O(n)$). We replace this with parallel reachability. By utilizing hopsets to restrict the graph diameter to $n^{1/2+o(1)}$, we can bypass sequential path-finding.

Algorithm 1 SmallCutFF(G, s, t, k): check if the minimum cut between s and t has size $< k$

Initialize flow $f(u, v) \leftarrow 0$ for all edges of the split graph.
for $i \leftarrow 1$ **to** k **do**
  $R_s, P_s \leftarrow$ ParallelReach(G_f, s)
  if $t \notin R_s$ **then**
    return `true` ▷ minimum cut is $< k$
  end if
  Let p be the augmenting path from s to t in P_s.
  Augment flow along p.
end for
return `false` ▷ minimum cut is at least k

Lemma 4.1. SmallCutFF(G, s, t, k) *runs in* $\widetilde{O}(mk)$ *work and* $n^{1/2+o(1)}$ *depth.*

Proof. By utilizing hopsets to restrict the graph diameter to $n^{1/2+o(1)}$, we can find up to k augmenting paths concurrently using standard parallel blocking flow techniques, structurally bypassing the need to find them strictly sequentially. Thus, the algorithm requires $\widetilde{O}(mk)$ work and exactly $n^{1/2+o(1)}$ depth. □

4.3 Modified local vertex cut

When the small side has volume at most α, we run the sequential-style LocalVC, whose depth is $O(\nu k)$. When the small side has volume larger than α, we replace DFS by ParallelReach. To preserve strict work bounds, we employ an **early abort filter**: if a sampled vertex belongs to the large side R, the first reachability call will immediately exceed the volume limit $k\nu$, allowing us to abort before executing the remaining k path reversals.

Algorithm 2 LocalVCReach(G, x, ν, k)

Input: G, x, ν, k
Output: a set L' containing x with $|N_G^{\text{out}}(L')| < k$ and $\text{vol}(L') < \nu$, or $\perp$
for $j \leftarrow 0$ **to** $\log n$ **do**
  $R_x, P_x \leftarrow$ ParallelReach(G, x)
  if $|R_x| \geq k\nu$ **then**
    return $\perp$ ▷ Early abort filter: sample hit the large side
  end if
  for $i \leftarrow 1$ **to** k **do**
    if $|R_x| < k\nu$ and $|N_G^{\text{out}}(R_x)| < k$ **then**
      return $N_G^{\text{out}}(R_x)$
    end if
    Sample a random edge $e = (y', y)$ from R_x
    Reverse the path P_{xy} and recompute R_x
  end for
end for
return $\perp$

Lemma 4.2. LocalVCReach(G, x, ν, k) *runs in* $\widetilde{O}(mk)$ *worst-case work, but* $\widetilde{O}(m)$ *work if it aborts early, and* $n^{1/2+o(1)}$ *depth.*

Proof. A call to ParallelReach costs $\widetilde{O}(m)$ work and $n^{1/2+o(1)}$ depth. If the algorithm does not abort early, the inner loop runs k times, yielding $\widetilde{O}(mk)$ work and $n^{1/2+o(1)}$ depth. If the early abort filter triggers, it executes only one reachability call, costing $\widetilde{O}(m)$ work. □

4.4 Main algorithm

Algorithm 3 ParallelVertexCut(G, k, α)

Input: $G = (V, E)$, k, $\alpha \in [\sqrt{n}, n]$
Output: a separator S with $|S| < k$, or $\perp$
$n \leftarrow |V|$, $m \leftarrow |E|$
for $i \leftarrow 1$ **to** $k \log n$ **do**
  Sample a random pair of edges $e = (x, x')$, $f = (y, y')$
  for $(s, t) \in \{(x, y), (x', y'), (x', y), (x, y')\}$ **do**
    if SmallCutFF(G, s, t, k) **then**
      $R_s \leftarrow$ ParallelReach(G_f, s)
      return $N_G^{\text{out}}(R_s)$
    end if
  end for
end for
for $i \leftarrow 1$ **to** $\log(m/k)$ **do**
  if $2^i < \alpha$ **then**
    for $j \leftarrow 1$ **to** $m \log n / 2^i$ **do**
      Sample a random edge $e = (x, y)$
      for $v \in \{x, y\}$ **do**
        $L' \leftarrow$ LocalVC$(G, v, 2^i, k)$
        if $L' \neq \perp$ **then**
          return $N_G^{\text{out}}(L')$
        end if
      end for
    end for
  else
    for $j \leftarrow 1$ **to** $m \log n / 2^i$ **do**
      Sample a random edge $e = (x, y)$
      for $v \in \{x, y\}$ **do**
        $L' \leftarrow$ LocalVCReach$(G, v, 2^i, k)$
        if $L' \neq \perp$ **then**
          return $N_G^{\text{out}}(L')$
        end if
      end for
    end for
  end if
end for
return $\perp$

5 Analysis

5.1 Balanced case

Assume $\mathrm{vol}^*_G(L) \geq m/k$. The probability that one sampled edge lies in $E^*(L,S)$ and the other in $E^*(R,S)$ is

$$\Pr(e \in E^*(L,S), f \in E^*(R,S)) = \frac{\mathrm{vol}^*_G(L)}{m} \cdot \frac{\mathrm{vol}^*_G(R)}{m} > \frac{1}{k},$$

using $\mathrm{vol}^*_G(L) + \mathrm{vol}^*_G(R) = \Omega(m)$ and $\mathrm{vol}^*_G(L) \geq m/k$. Repeating $k \log n$ times succeeds w.h.p. Each iteration tests four source–sink pairs, and each test calls SmallCutFF, which costs $\widetilde{O}(mk)$ work. Thus, the balanced phase costs

$$W_1 = \widetilde{O}(mk^2) \text{ work}, \qquad D_1 = n^{1/2+o(1)} \text{ depth}.$$

5.2 Unbalanced case

Assume $\mathrm{vol}^*_G(L) < m/k$. There is an integer i with $\mathrm{vol}^*_G(L) \in [2^{i-1}, 2^i]$, and sampling $\widetilde{O}(m/2^i)$ edges hits $E^*(L,S)$ w.h.p. We split into two subcases.

Small volume ($2^i < \alpha$). We use LocalVC, which costs $\widetilde{O}(2^i k^2)$ per call. The total work is

$$W_{21} = \sum_{i=1}^{\log \alpha} \widetilde{O}(2^i k^2) \cdot \frac{m}{2^i} \log n = \widetilde{O}(mk^2),$$

and the depth is $D_{21} = \max_{0 \leq i \leq \log \alpha}\{2^i k\} = \alpha k$.

Large volume ($2^i \geq \alpha$). We use LocalVCReach. Because of the early abort filter, a sample that hits the large side R aborts after a single ParallelReach call, costing only $\widetilde{O}(m)$ work. The full $\widetilde{O}(mk)$ cost is only paid by the successful sample, which is absorbed by the W_1 bound. Thus, the expected work per sample is $\widetilde{O}(m)$.

$$W_{22} = \sum_{i=\log \alpha}^{\log(m/k)} \widetilde{O}(m) \frac{m}{2^i} \log n = \tilde{O}(m^2) \sum_{i=\log \alpha}^{\log(m/k)} \frac{1}{2^i}$$

Calculating the geometric series $S = \sum_{i=\log \alpha}^{\log(m/k)} \frac{1}{2^i}$:

$$S = \left(\frac{2}{2^{\log \alpha}}\right) - \left(\frac{2}{2^{\log m/k+1}}\right) = \frac{2}{\alpha} - \frac{k}{m}$$

Multiplying by m^2:

$$m^2 S = m^2 \left(\frac{2}{\alpha} - \frac{k}{m}\right) = \frac{2m^2}{\alpha} - mk$$

Since $k = O(\sqrt{n})$, this yields:

$$W_{22} = \tilde{O}\left(\frac{m^2}{\alpha}\right).$$

The depth strictly utilizes the diameter bound $D_{22} = n^{1/2+o(1)}$.

5.3 Total

Combining the three parts,

$$W = W_1 + W_{21} + W_{22} = \widetilde{O}(mk^2) + \widetilde{O}(mk^2) + \widetilde{O}\left(\frac{m^2}{\alpha}\right) = \widetilde{O}\left(mk^2 + \frac{m^2}{\alpha}\right),$$

and

$$D = \max\{D_1, D_{21}, D_{22}\} = \max\{n^{1/2+o(1)}, \alpha k\}.$$

This proves Theorem 1.1.

6 Constant separator size

When $k = O(1)$, if we revisit the edge-sampling tradeoff:

$$W_{\text{edge}} = \widetilde{O}\left(\frac{m^2}{\alpha}\right), \qquad D_{\text{edge}} = \max\{n^{1/2+o(1)}, \alpha\}.$$

Restricting $\alpha \in [\sqrt{n}, n]$ gives $W_{\text{edge}} = \widetilde{O}(m^2/\alpha)$ and $D_{\text{edge}} = O(\alpha)$.

For vertex sampling, we replace edge sampling by vertex sampling and use a variant LocalVCVertex that runs in $\widetilde{O}(\nu^2 k^3)$ sequential time. Running this for n/ν samples over all $\nu = 2^i$ up to $\log \alpha$:

$$W_{21} = \sum_{i=1}^{\log \alpha} \tilde{O}(n 2^i k^3) = \tilde{O}(n\alpha k^3)$$

For LocalVCReach with vertex sampling, the number of samples is $n/2^i$:

$$W_{22} = \sum_{i=\log \alpha}^{\log(n/k)} \tilde{O}(m)\frac{n}{2^i}\log n = \tilde{O}(mn)\left(\frac{2}{\alpha} - \frac{k}{n}\right) = \tilde{O}\left(\frac{mn}{\alpha}\right)$$

The resulting work is:

$$W_{\text{vertex}} = \widetilde{O}\left(mk^2 + n\alpha k^3 + \frac{mn}{\alpha}\right).$$

For $k = O(1)$, this evaluates to $\widetilde{O}(m + n\alpha + mn/\alpha)$. As with edge sampling, we restrict values for $\alpha \in [\sqrt{n}, n]$, meaning this simplifies to $W_{\text{vertex}} = \widetilde{O}(mn/\alpha)$ with depth $D_{\text{vertex}} = O(\alpha)$. This concludes the proof of Theorem 1.2.

7 Conclusion and future work

We presented parallel algorithms for small vertex cut in the PRAM model, giving a work–depth tradeoff. The main technical ingredients are a parallel Ford–Fulkerson subroutine based on parallel reachability and a modified local vertex cut procedure that replaces depth-heavy DFS with hopsets and shortcuts. Several directions remain open: whether an $O(\nu k)$-time LocalVC exists, whether the $O(n^3)$ bound can be broken for $k = \Omega(n)$, extension to vertex-weighted graphs, removal of the $n^{o(1)}$ term in the depth of parallel reachability, and breaking the $\sqrt{n}$ depth barrier. We conjecture that there exists a parallel algorithm that, given x, μ, explores μ vertices reachable from x in $O(\mu)$ work and $O(\sqrt{\mu})$ depth.

References

[1] Joseph Cheriyan and Ramakrishna Thurimella. Algorithms for parallel k-vertex connectivity and sparse certificates. In *Proceedings of the Twenty-Third Annual ACM Symposium on Theory of Computing*, STOC '91, page 391–401, New York, NY, USA, 1991. Association for Computing Machinery.

[2] Jeremy T. Fineman. Nearly work-efficient parallel algorithm for digraph reachability. *SIAM Journal on Computing*, 49(5):STOC18–500–STOC18–539, 2018.

[3] Danupon Nanongkai, Thatchaphol Saranurak, and Sorrachai Yingchareonthawornchai. Breaking quadratic time for small vertex connectivity and an approximation scheme. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2019, page 241–252, New York, NY, USA, 2019. Association for Computing Machinery.

[4] Aaron Sidford, Arun Jambulapati, and Yang P. Liu. Parallel reachability in almost linear work and square root depth. *2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1664–1686, 2019.